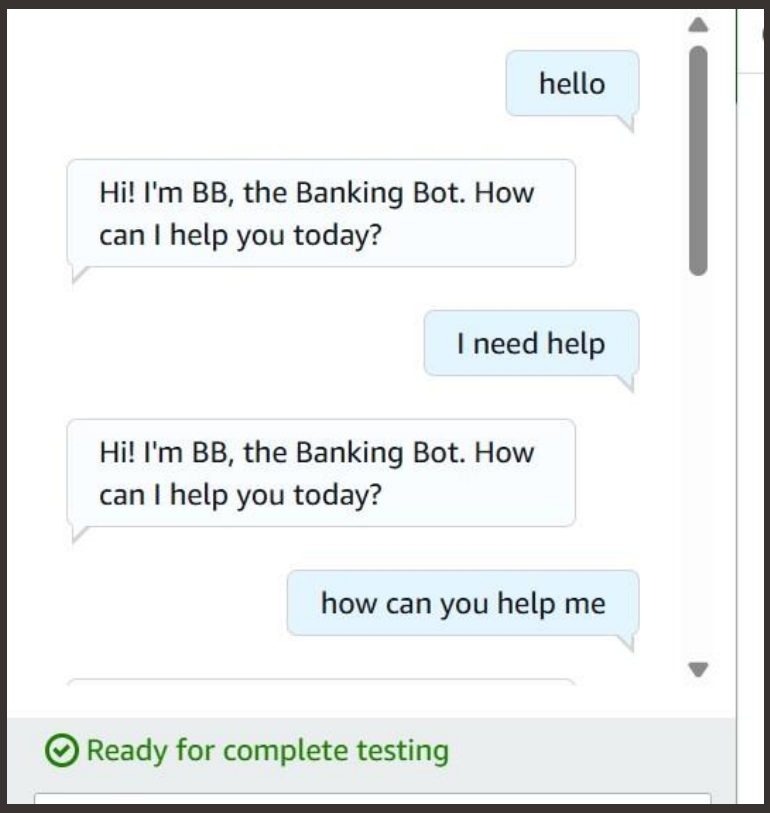
Build a Chatbot with Amazon Lex

Danford Amos



Introducing Today's Project!

Project overview

In this project, I will build a chatbot using Amazon Lex and the AWS Console, covering the full workflow from design to deployment. I will model intents, define slots, configure dialog flows, and integrate the bot into a real-world application. The goal is to strengthen my practical understanding of applied AI within AWS.

What is Amazon Lex?

Amazon Lex is an AWS service for quickly building conversational chatbots using built-in NLU and speech recognition.

Key tools and concepts

I learned how to use Amazon Lex to build a chatbot using intents, utterances, slots, and the FallbackIntent, along with customizing responses for a more natural conversation.

Personal reflections

I was surprised by how quickly I could build a working chatbot with Amazon Lex and how much customizing the FallbackIntent and response variations improved the overall conversational experience.

Project timeline

This project took me approximately a few hours to complete. The most challenging part was understanding how Amazon Lex handles intent recognition and fallback behavior. It was most rewarding to see the chatbot respond naturally after customizing the FallbackIntent and adding response variations.

Setting up a Lex chatbot

Approach to chatbot setup

In this step I will create a new bot from scratch, since Amazon Lex gives me full control to customize a banking-style chatbot to fit my design requirements.

Chatbot creation process

I created my chatbot from scratch using Amazon Lex, and the initial setup took only seven minutes.

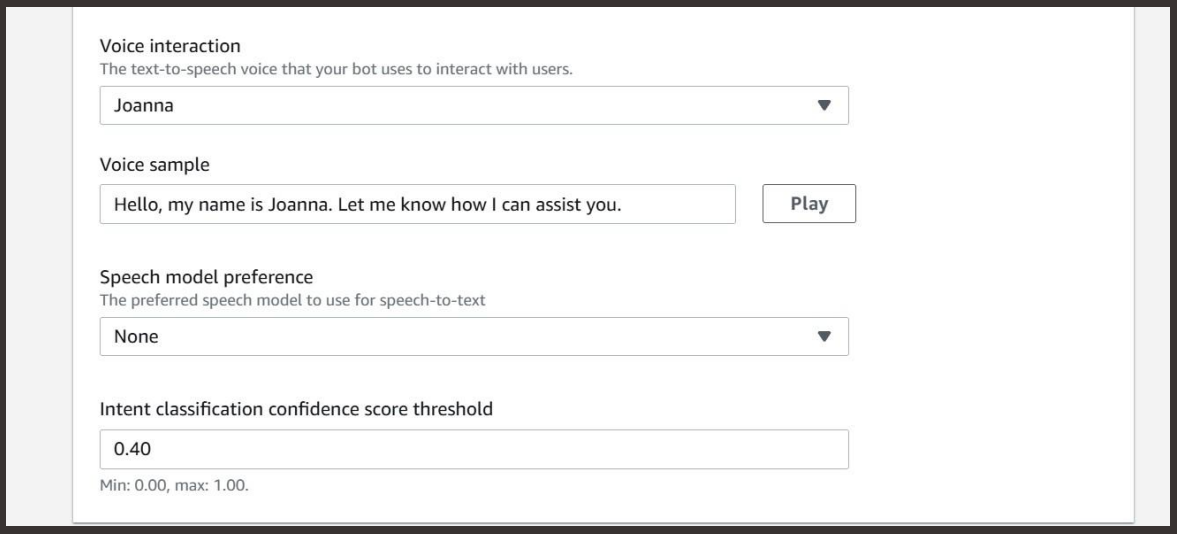
IAM role configuration

While creating my chatbot, I also created an IAM role with basic permissions, since Amazon Lex requires authorization to interact with other AWS services such as Lambda.

Intent classification confidence score

For intent classification, I kept the default confidence threshold of 0.40, meaning the chatbot must be at least 40% confident in its prediction before responding to a user’s input.

Danford Amos



Intents

In this step, I will train my bot to greet users so the chatbot provides a friendly and welcoming experience.

Creating my first intent

Intents represent the user's goals—the actions or outcomes the chatbot must understand and respond to.

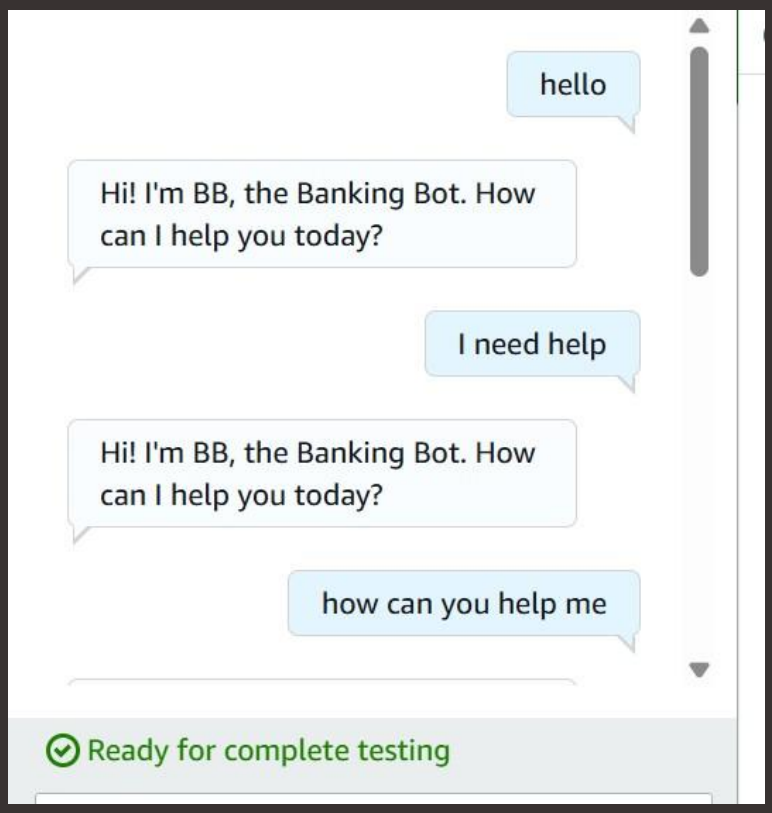
My first intent, Welcome Intent, is designed to greet users when they say hello or ask for help.

Testing chatbot responses

I launched and tested my chatbot to validate the intent-recognition pipeline, and the model successfully mapped user utterances such as "Hiya" and "Hello" to the WelcomeIntent. This confirms that Lex's NLU engine correctly generalized from the sample utterances I provided during training and is able to classify semantically similar greetings with the expected confidence threshold.

Handling unrecognized inputs

When I tested my chatbot with utterances like “How are you?” and “Good morning,” Lex returned the message “Intent FallbackIntent is fulfilled.” This occurred because the NLU engine could not map those inputs to any of the intents I initially defined. Since no intent met the minimum confidence threshold, Lex routed the request to the built-in FallbackIntent, which is triggered whenever the model cannot confidently classify a user utterance.



Configuring FallbackIntent

I will customize the FallbackIntent so the chatbot gives a friendly response whenever an utterance can’t be confidently classified.

When FallbackIntent triggers

FallbackIntent is a built-in Amazon Lex intent that triggers whenever the chatbot cannot recognize or confidently classify the user’s input.

I customized the FallbackIntent to ensure that when Lex’s NLU engine fails to achieve the minimum confidence score required to map an utterance to a defined intent, the chatbot still returns a controlled, user-friendly response. This allows the system to gracefully handle out-of-domain inputs while maintaining a consistent conversational experience, even during classification errors

My FallbackIntent configuration

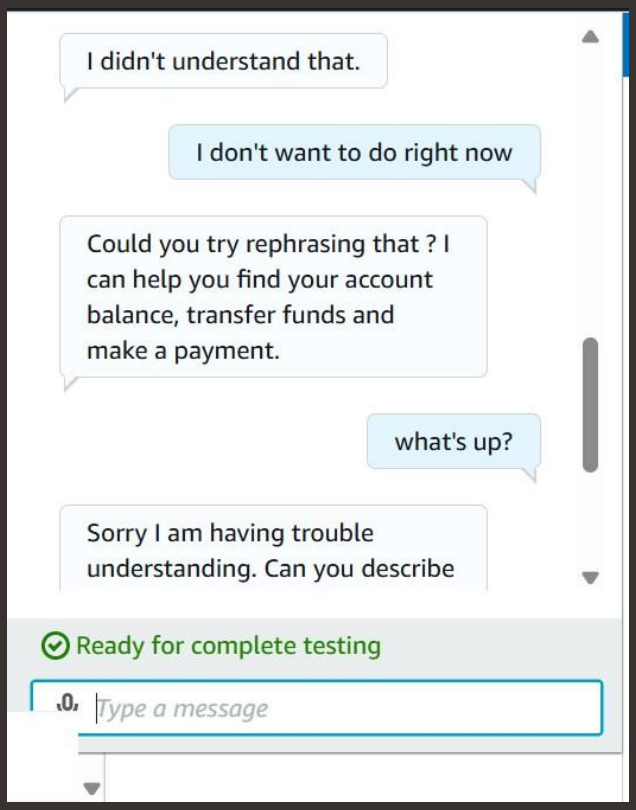
To configure the FallbackIntent, I customized the closing response messages and added multiple variations so the chatbot can return more natural-sounding, conversational replies during fallback scenarios.

placeholder

I also added multiple response variations.

This ensures that when the chatbot triggers the FallbackIntent, end users receive slightly different friendly messages instead of the same repeated response, making the interaction feel more natural even when the system cannot interpret their input.

FallbackIntent with Variations



Initial Responses

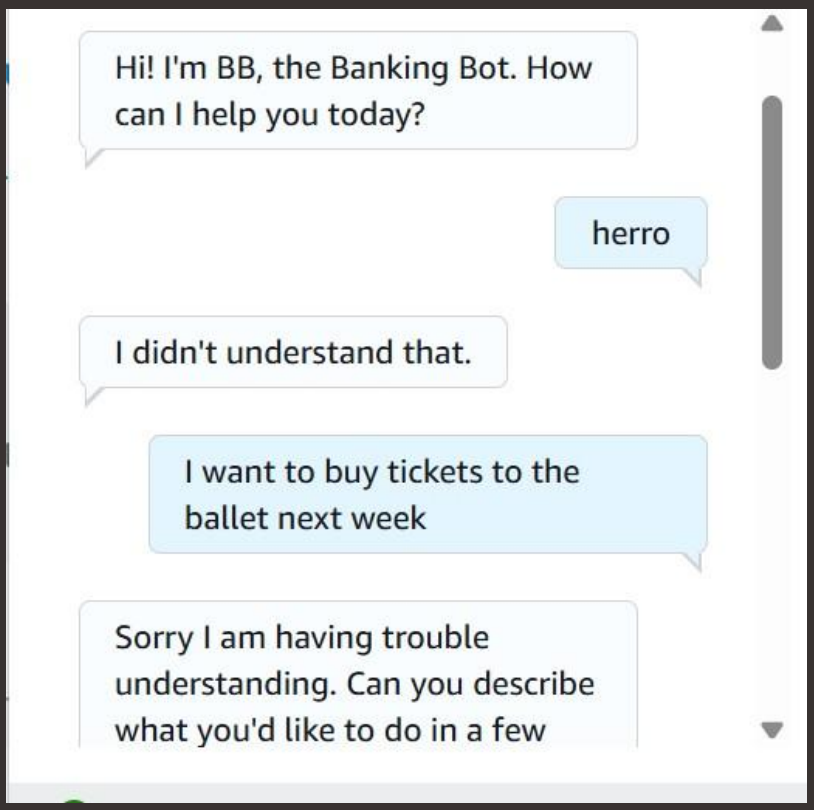
In this project extension, I will enhance the FallbackIntent by defining an initial response and adding additional variations. This helps me deepen my familiarity with Amazon Lex's response-generation framework while improving the chatbot's ability to deliver natural, user-friendly messages when it cannot interpret an input.

Setting up initial responses

For this extension, I configured the FallbackIntent to send an initial response. I added variations such as "Hmmm, this is interesting..." and "One moment..." so the chatbot provides a more human-like, conversational reaction whenever it detects the fallback intent. This gives end users a smoother experience by making the system feel less rigid when it cannot interpret their input.

Initial response implementation

The initial response messages I configured—"Hmmm, this is interesting..." and "One moment..."—give the chatbot a more conversational feel. For the user, this means the bot provides a natural, human-like reaction before delivering the final fallback message.


Hi! I'm BB, the Banking Bot. How can I help you today?
herro
I didn't understand that.
I want to buy tickets to the ballet next week
Sorry I am having trouble understanding. Can you describe what you'd like to do in a few